

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2015 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 3 (15 iulie 2015)	2
SUBIECTUL I (30 de puncte)	2
1. Structura de date: Arbori Binari	2
2. Rețele: Serviciul WWW (World Wide Web)	3
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Cuvinte și șirul lor mijlociu	3
2. Algoritm eficient: Diferență simetrică impare ($O(N_a + N_b)$ timp)	5
SUBIECTUL al III-lea (30 de puncte)	8
1. Exercițiul ca metodă didactică pentru algoritmi elementari	8
2. Portofoliu pentru secvența B: baze de date	8

I. Rezolvare Titularizare Varianta 3 (15 iulie 2015)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Structura de date: Arbori Binari

- **Definiții:** Un arbore binar este o structură de date ierarhică formată din noduri, în care fiecare nod are cel mult doi descendenți, numiți fiul stâng și fiul drept.
- **Parcurgeri:**
 - **Preordine** (Rădăcină-Stânga-Dreapta).
 - **Inordine** (Stânga-Rădăcină-Dreapta).
 - **Postordine** (Stânga-Dreapta-Rădăcină).
- **Reprezentare statică** (vectori de tați/fii) vs **Reprezentare dinamică** (pointeri/adrese).
- **Problemă 1 - reprezentare statică:** Se dă un arbore binar cu n noduri, memorat prin doi vectori $st[]$ și $dr[]$, unde $st[i]$ este fiul stâng al nodului i , iar $dr[i]$ este fiul drept, valoarea 0 indicând lipsa fiului. Să se determine numărul frunzelor.

Descriere: Un nod este frunză dacă nu are nici fiu stâng, nici fiu drept. Parcurgem toate nodurile și numărăm pozițiile i pentru care $st[i] = 0$ și $dr[i] = 0$.

```
#include <iostream>
using namespace std;

int main() {
    int n, st[101], dr[101], nr = 0;
    cin >> n;
    for (int i = 1; i <= n; ++i) {
        cin >> st[i] >> dr[i];
    }
    for (int i = 1; i <= n; ++i) {
        if (st[i] == 0 && dr[i] == 0) nr++;
    }
    cout << nr;
    return 0;
}
```

- **Problemă 2 - reprezentare dinamică:** Se dă rădăcina unui arbore binar memorat prin noduri alocate dinamic. Să se determine recursiv numărul frunzelor.

Descriere: Dacă arborele este vid, nu are frunze. Dacă nodul curent nu are descendenți, el contribuie cu o frunză. Altfel, rezultatul este suma frunzelor din subarboarele stâng și subarboarele drept.

```
#include <iostream>
using namespace std;
struct Nod {
    int info;
    Nod *st, *dr;
};

int nrFrunze(Nod* rad) {
    if (rad == nullptr) return 0;
    if (rad->st == nullptr && rad->dr == nullptr) return 1;
    return nrFrunze(rad->st) + nrFrunze(rad->dr);
}
```

2. Rețele: Serviciul WWW (World Wide Web)

- **Concepte:** Rețea (echipamente interconectate), Internet (rețeaua globală), TCP (protocol de transport orientat pe conexiune și sigur).
- **Model WWW:** Model client-server, protocolul HTTP (HyperText Transfer Protocol), Web Browser (aplicație client).
- **Organizare:** Pagini HTML, site-uri web (grup de pagini web sub același domeniu), portal (punct de acces centralizator).
- **Adresare/Căutare:** URL (Uniform Resource Locator), motoare de căutare (ex. Google).

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Cuvinte și șirul lor mijlociu

Soluție C++:

```
#include <iostream>
#include <string>
#include <sstream>
#include <vector>
#include <map>
using namespace std;

bool isVowel(char c) {
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
}

void mijlociu(const string &s, string &sm) {
    sm = "";
    if (s.empty()) return;
    sm += s[0];
    for (int i = 1; i < s.length(); ++i) {
        char a = s[i-1];
        char b = s[i];
        if (isVowel(a) && isVowel(b) && a != b) {
            char m = (char)((a + b) / 2);
            sm += m;
        }
        sm += b;
    }
}

int main() {
    string text;
    if (!getline(cin, text)) return 0;

    stringstream ss(text);
    string cuvant;
    vector<string> cuvinte;
    while (ss >> cuvant) {
        cuvinte.push_back(cuvant);
    }
}
```

```

int n = cuvinte.size();
vector<string> transformed(n);
for (int i = 0; i < n; ++i) {
    mijlociu(cuvinte[i], transformed[i]);
}

int count = 0;
for (int i = 0; i < n; ++i) {
    for (int j = i + 1; j < n; ++j) {
        if (transformed[i] == transformed[j]) {
            count++;
        }
    }
}

cout << count << endl;
return 0;
}

```

Soluție Pascal:

```

program PerechiMijlocii;
var
    text_input, cuvant: string;
    cuvinte, transformed: array[1..100] of string;
    nr_cuvinte, i, j, count: integer;

function isVowel(c: char): boolean;
begin
    isVowel := (c = 'a') or (c = 'e') or (c = 'i') or (c = 'o') or (c = 'u');
end;

procedure mijlociu(s: string; var sm: string);
var
    idx, char_code: integer;
    a, b, m: char;
begin
    sm := '';
    if length(s) = 0 then exit;
    sm := sm + s[1];
    for idx := 2 to length(s) do
        begin
            a := s[idx - 1];
            b := s[idx];
            if isVowel(a) and isVowel(b) and (a <> b) then
                begin
                    char_code := (ord(a) + ord(b)) div 2;
                    m := chr(char_code);
                    sm := sm + m;
                end;
            sm := sm + b;
        end;
    end;

begin
    readln(text_input);
    text_input := text_input + ' ';

```

```

nr_cuvinte := 0;
cuvant := '';
for i := 1 to length(text_input) do
begin
    if text_input[i] <> ' ' then
        cuvant := cuvant + text_input[i]
    else
        begin
            if length(cuvant) > 0 then
                begin
                    nr_cuvinte := nr_cuvinte + 1;
                    cuvinte[nr_cuvinte] := cuvant;
                    mijlociu(cuvant, transformed[nr_cuvinte]);
                    cuvant := '';
                end;
            end;
        end;
end;

count := 0;
for i := 1 to nr_cuvinte do
begin
    for j := i + 1 to nr_cuvinte do
    begin
        if transformed[i] = transformed[j] then
            count := count + 1;
        end;
    end;
end;
writeln(count);
end.

```

2. Algoritm eficient: Diferență simetrică impare ($O(n_a + n_b)$ timp)

Metoda: Primul șir este descrescător, iar al doilea este crescător. Pentru a obține o parcurgere comună în ordine crescătoare, memorăm numerele impare distincte din primul șir și le inversăm. Din al doilea șir memorăm direct numerele impare distincte, deoarece este deja crescător. Apoi aplicăm interclasarea a două șiruri sortate: valorile mai mici apar numai într-un șir și se afișează, valorile egale se sar deoarece apar în ambele șiruri și nu aparțin diferenței simetrice.

Eficiență: Fiecare element din cele două șiruri este citit o singură dată, iar interclasarea avansează cel puțin un indice la fiecare pas. Complexitatea este $O(n_a + n_b)$ timp; memoria folosită este liniară în numărul valorilor impare distincte memorate.

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("titu.in");
    int na, nb;
    if (!(fin >> na >> nb)) return 0;

```

```

vector<long long> A, B;
for (int i = 0; i < na; ++i) {
    long long val;
    fin >> val;
    if (val % 2 != 0) {
        if (A.empty() || val != A.back()) A.push_back(val);
    }
}
reverse(A.begin(), A.end());

for (int i = 0; i < nb; ++i) {
    long long val;
    fin >> val;
    if (val % 2 != 0) {
        if (B.empty() || val != B.back()) B.push_back(val);
    }
}
fin.close();

int i = 0, j = 0;
vector<long long> rez;
while (i < A.size() && j < B.size()) {
    if (A[i] < B[j]) {
        rez.push_back(A[i]);
        i++;
    } else if (B[j] < A[i]) {
        rez.push_back(B[j]);
        j++;
    } else {
        long long val = A[i];
        while (i < A.size() && A[i] == val) i++;
        while (j < B.size() && B[j] == val) j++;
    }
}
while (i < A.size()) rez.push_back(A[i++]);
while (j < B.size()) rez.push_back(B[j++]);

if (rez.empty()) {
    cout << "nu exista\n";
} else {
    for (int k = 0; k < rez.size(); ++k) {
        cout << rez[k] << (k == rez.size() - 1 ? "" : " ");
    }
    cout << "\n";
}
return 0;
}

```

Soluție Pascal:

```

program DiferentaSimetrica;
var
    fin: text;
    na, nb, i, j, k, count_a, count_b, count_rez: integer;
    val: int64;
    A_desc, A, B: array[1..10000] of int64;

```

```

rez: array[1..20000] of int64;
begin
  assign(fin, 'titu.in'); reset(fin);
  read(fin, na, nb);
  count_a := 0;
  for i := 1 to na do
  begin
    read(fin, val);
    if val mod 2 <> 0 then
    begin
      if (count_a = 0) or (val <> A_desc[count_a]) then
      begin
        count_a := count_a + 1; A_desc[count_a] := val;
      end;
    end;
  end;
  for i := 1 to count_a do A[i] := A_desc[count_a - i + 1];

  count_b := 0;
  for i := 1 to nb do
  begin
    read(fin, val);
    if val mod 2 <> 0 then
    begin
      if (count_b = 0) or (val <> B[count_b]) then
      begin
        count_b := count_b + 1; B[count_b] := val;
      end;
    end;
  end;
  close(fin);

  i := 1; j := 1; count_rez := 0;
  while (i <= count_a) and (j <= count_b) do
  begin
    if A[i] < B[j] then
    begin
      count_rez := count_rez + 1; rez[count_rez] := A[i]; i := i + 1;
    end
    else if B[j] < A[i] then
    begin
      count_rez := count_rez + 1; rez[count_rez] := B[j]; j := j + 1;
    end
    else
    begin
      val := A[i];
      while (i <= count_a) and (A[i] = val) do i := i + 1;
      while (j <= count_b) and (B[j] = val) do j := j + 1;
    end;
  end;
  while i <= count_a do begin count_rez := count_rez + 1; rez[count_rez] := A[i];
i := i + 1; end;
  while j <= count_b do begin count_rez := count_rez + 1; rez[count_rez] := B[j];
j := j + 1; end;

  if count_rez = 0 then writeln('nu exista')

```

```

else
begin
  for k := 1 to count_rez do
  begin
    write(rez[k]); if k < count_rez then write(' ');
  end;
  writeln;
end;
end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Exercițiul ca metodă didactică pentru algoritmi elementari

Caracteristici: presupune repetarea conștientă a unor operații; fixează deprinderi algoritmice; permite gradarea dificultății. **Tipuri:** exerciții de recunoaștere, exerciții de aplicare, exerciții de creație.

Activitatea 1: calculul unei sume de expresii simple.

Scenariu didactic: Profesorul pornește de la expresia $1 + 2 + \dots + n$, reactualizează noțiunile de variabilă contor și acumulator, apoi cere elevilor să propună pașii algoritmului în pseudocod. Elevii rezolvă mai întâi manual pentru $n=5$, apoi scriu structura repetitivă pentru $i \leftarrow 1..n$. Profesorul verifică inițializarea sumei cu 0, iar elevii rulează algoritmul pentru valori diferite și compară rezultatul cu formula matematică.

Activitatea 2: determinarea produsului numerelor pare dintr-o secvență.

Scenariu didactic: Profesorul prezintă o secvență mixtă de valori și cere elevilor să marcheze numai termenii pari. Elevii identifică testul $x \bmod 2 = 0$, apoi discută de ce produsul se inițializează cu 1, nu cu 0. Elevii implementează algoritmul, testează cazul în care nu există numere pare și decid mesajul afișat în această situație. Profesorul insistă asupra corectitudinii inițializărilor și a separării citirii de prelucrare.

2. Portofoliu pentru secvența B: baze de date

Scop: evaluarea progresului elevului în proiectarea și utilizarea unei baze de date simple.

Elemente: schema conceptuală, tabelele create, capturi cu cheia primară/indexul, set de înregistrări, interogări și reflecție personală.

Criterii produs: corectitudinea structurii tabelor; definirea cheii primare și a indexului; validitatea datelor introduse. **Criterii atitudinale:** consecvența în lucru; capacitatea de autoevaluare și îmbunătățire.